

Systemy Obliczeniowe

Laboratorium 6 – Lista A (12 grudnia 2024)

dr inż. Paweł Trajdos

1 Uwagi

1. Po zakończeniu realizacji zadań należy załadować na Eportal skrypy pythona. Format nazwy pliku `ZA.py` (A to numer zadania). Źle nazwane pliki nie są oceniane.
2. W przypadku niepewności lub zauważenia jakichkolwiek błędów w instrukcji należy niezwłocznie powiadomić prowadzącego laboratorium w celu wyjaśnienia sprawy. Reklamacje po zakończeniu zajęć nie będą uwzględniane.

2 Zadania

Zadanie 6.0(Pkt. 6.0):

Zadanie pierwsze polega na przeprowadzeniu prostego eksperymentu z wykorzystaniem algorytmu regresji logistycznej.

Na początek należy wybrać – spośród zbiorów udostępnionych na laboratorium pierwszym – pojedynczy zbiór danych znajdujący się w pliku CSV. Następnie należy wczytać go za pomocą poniższego kodu:

```
1 from pyspark.sql import SparkSession
2
3 spark = SparkSession.builder.appName("-").getOrCreate()
4
5 df = spark.read.csv("/content/sample_data/nazwa_zbioru.csv", inferSchema=True)
```

Dla pewności wyświetlmy wczytany zbiór danych za pomocą `df.show()`.

Modele uczenia maszynowego znajdujące się w bibliotece **Sparka** wymagają, aby zbiór danych zawierał kolumnę z wektorami wszystkich cech o nazwie **features** oraz kolumnę z etykietami klas o nazwie **label**. Operacji tych można dokonać odpowiednio za pomocą funkcji **withColumnRenamed**¹ oraz klasy **VectorAssembler**².

Przykład przekodowanego zbioru można zobaczyć na Rys 1

Ostatnią potrzebną operacją jest podział przygotowanych danych na zbiór treningowy oraz testowy (dokładnie po połowie) z wykorzystaniem funkcji **randomSplit()**³.

¹<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.DataFrame.withColumnRenamed.html>

²<https://spark.apache.org/docs/3.1.1/api/python/reference/api/pyspark.ml.feature.VectorAssembler.html>

³<https://spark.apache.org/docs/3.1.1/api/python/reference/api/pyspark.sql.DataFrame.randomSplit.html>

_c0	_c1	_c2	_c3	_c4	_c5	_c6	_c7	_c8	_c9	_c10	_c11	_c12	_c13	label	features
1	22.08	11.46	2	4	4	1.585	0	0	0	1	2	100	1213	0	[1.0,22.08,11.46,...
0	22.67	7.0	2	8	4	0.165	0	0	0	0	2	160	1	0	[0.0,22.67,7.0,2....
0	29.58	1.75	1	4	4	1.25	0	0	0	1	2	280	1	0	[0.0,29.58,1.75,1...
0	21.67	11.5	1	5	3	0.0	1	1	11	1	2	0	1	1	[0.0,21.67,11.5,1...
1	20.17	8.17	2	6	4	1.96	1	1	14	0	2	60	159	1	[1.0,20.17,8.17,2...
0	15.83	0.585	2	8	8	1.5	1	1	2	0	2	100	1	1	[0.0,15.83,0.585,...
1	17.42	6.5	2	3	4	0.125	0	0	0	0	2	60	101	0	[1.0,17.42,6.5,2...
0	58.67	4.46	2	11	8	3.04	1	1	6	0	2	43	561	1	[0.0,58.67,4.46,2...
1	27.83	1.0	1	2	8	3.0	0	0	0	0	2	176	538	0	[1.0,27.83,1.0,1...
0	55.75	7.08	2	4	8	6.75	1	1	3	1	2	100	51	0	[0.0,55.75,7.08,2...
1	33.5	1.75	2	14	8	4.5	1	1	4	1	2	253	858	1	[1.0,33.5,1.75,2...
1	41.42	5.0	2	11	8	5.0	1	1	6	1	2	470	1	1	[1.0,41.42,5.0,2...
1	20.67	1.25	1	8	8	1.375	1	1	3	1	2	140	211	0	[1.0,20.67,1.25,1...
1	34.92	5.0	2	14	8	7.5	1	1	6	1	2	0	1001	1	[1.0,34.92,5.0,2...
1	58.58	2.71	2	8	4	2.415	0	0	0	1	2	320	1	0	[1.0,58.58,2.71,2...
1	48.08	6.04	2	4	4	0.04	0	0	0	0	2	0	2691	1	[1.0,48.08,6.04,2...
1	29.58	4.5	2	9	4	7.5	1	1	2	1	2	330	1	1	[1.0,29.58,4.5,2...
0	18.92	9.0	2	6	4	0.75	1	1	2	0	2	88	592	1	[0.0,18.92,9.0,2...
1	20.0	1.25	1	4	4	0.125	0	0	0	0	2	140	5	0	[1.0,20.0,1.25,1...
0	22.42	5.665	2	11	4	2.585	1	1	7	0	2	129	3258	1	[0.0,22.42,5.665,...

only showing top 20 rows

Rysunek 1: Przykładowe wyniki.

Kolejnym krokiem jest zdefiniowanie klasyfikatora Random Forest (**RandomForest**⁴), a następnie zdefiniowanie dwóch wartości wybranego przez siebie parametru klasyfikatora, które zostaną sprawdzone w eksperymencie (****ParamGridBuilder()**⁵). W końcu, za pomocą klasy **TrainValidationSplit**⁶ dokonujemy optymalizacji modelu na pojedynczym podziale, gdzie trening dokonywany jest na 80% zbioru treningowego, a walidacja na 20%. Przydatny przykład można znaleźć **tutaj**⁷.

Po przeprowadzeniu treningu, należy wyświetlić dla pierwszych 20 instancji zbioru testowego – z wykorzystaniem najlepszego modelu – kolejno ich cechy, otrzymane wsparcia, otrzymane predykcje oraz prawdziwe etykiety. Przykład na Rys 2

Zadanie 6.1(Pkt. 6.0):

W zadaniu drugim należy powtórzyć eksperyment z zadania pierwszego, uwzględniając następujące zmiany:

- Jako klasyfikator należy wykorzystać naiwny klasyfikator bayesowski (**NaiveBayes**⁸).
- Należy przeprowadzić szerszy przegląd wartości hiperparametrów (dwa parametry po trzy wartości każdy).
- Wyboru najlepszego modelu należy dokonać za pomocą pięciokrotnej walidacji krzyżowej (**CrossValidator**⁹).

⁴<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.mllib.tree.RandomForest.html>

⁵<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.tuning.ParamGridBuilder.html>

⁶<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.tuning.TrainValidationSplit.html>

⁷<https://spark.apache.org/docs/latest/ml-tuning.html>

⁸<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.classification.NaiveBayes.html>

⁹<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.tuning.CrossValidator.html>

features	probability	label	prediction
[0.0,16.0,0.165,2.0,6.0,4.0,1.0,0.0,1.0,2.0,1.0,2.0,320.0,2.0]	[0.8137803068771071,0.18629969312289285]	0	0.0
[0.0,16.08,0.335,2.0,1.0,1.0,0.0,0.0,1.0,1.0,0.0,2.0,160.0,127.0]	[0.9188857003751382,0.08111429962486183]	0	0.0
[0.0,16.33,0.21,2.0,6.0,4.0,0.125,0.0,0.0,0.0,0.0,2.0,200.0,2.0]	[0.8849767807980882,0.11502321920991176]	0	0.0
[0.0,17.92,0.54,2.0,8.0,4.0,1.75,0.0,1.0,1.0,1.0,2.0,80.0,6.0]	[0.758604849331913,0.24139515066808703]	0	0.0
[0.0,18.08,0.375,3.0,13.0,1.0,10.0,0.0,0.0,0.0,1.0,1.0,300.0,1.0]	[0.6164229302664657,0.38357706973353434]	1	0.0
[0.0,18.17,10.0,1.0,11.0,8.0,0.165,0.0,0.0,0.0,0.0,2.0,340.0,1.0]	[0.7924371753145786,0.20756282468342142]	0	0.0
[0.0,18.25,10.0,2.0,9.0,4.0,1.0,0.0,1.0,1.0,0.0,2.0,120.0,2.0]	[0.7270447604458794,0.2729552395541206]	0	0.0
[0.0,18.42,9.25,2.0,11.0,4.0,1.21,1.0,1.0,4.0,0.0,2.0,60.0,541.0]	[0.22567049434123723,0.7743295056587628]	1	1.0
[0.0,18.83,4.415,1.0,8.0,8.0,3.0,1.0,0.0,0.0,0.0,2.0,240.0,1.0]	[0.458178772478354,0.5418212227521646]	1	1.0
[0.0,18.92,9.0,2.0,6.0,4.0,0.75,1.0,1.0,2.0,0.0,2.0,88.0,592.0]	[0.3848757616467515,0.6151242383532485]	1	1.0
[0.0,18.92,9.25,1.0,8.0,4.0,1.0,1.0,1.0,4.0,1.0,2.0,80.0,501.0]	[0.3744490589370919,0.6255509410629881]	1	1.0
[0.0,19.5,0.165,2.0,11.0,4.0,0.04,0.0,0.0,0.0,1.0,2.0,380.0,1.0]	[0.8213803156788951,0.17861968432110487]	0	0.0
[0.0,19.75,0.75,2.0,8.0,4.0,0.795,1.0,1.0,5.0,1.0,2.0,140.0,6.0]	[0.31422255637621427,0.6857774436237850]	0	1.0
[0.0,20.08,0.125,2.0,11.0,4.0,1.0,0.0,1.0,1.0,0.0,2.0,240.0,769.0]	[0.7106567483275646,0.28934325167243535]	1	0.0
[0.0,20.33,10.0,2.0,8.0,8.0,1.0,1.0,1.0,4.0,0.0,2.0,50.0,1466.0]	[0.20570771374581958,0.7942122862541804]	1	1.0
[0.0,20.42,10.5,1.0,14.0,8.0,0.0,0.0,0.0,0.0,1.0,2.0,154.0,33.0]	[0.7075424302619451,0.2924575697380549]	0	0.0
[0.0,20.5,11.835,2.0,8.0,8.0,6.0,1.0,0.0,0.0,0.0,2.0,340.0,1.0]	[0.3148929425454568,0.6851070574545433]	1	1.0
[0.0,20.75,10.25,2.0,11.0,4.0,0.71,1.0,1.0,2.0,1.0,2.0,49.0,1.0]	[0.25425031954248994,0.7457496804575101]	1	1.0
[0.0,20.75,10.335,2.0,13.0,8.0,0.335,1.0,1.0,1.0,1.0,2.0,80.0,51.0]	[0.166165629106577,0.833834370893423]	1	1.0
[0.0,20.83,3.0,2.0,6.0,4.0,0.04,1.0,0.0,0.0,0.0,2.0,100.0,1.0]	[0.5583862553441677,0.44161374465583225]	1	0.0

only showing top 20 rows

Rysunek 2: Przykładowe wyniki.

html?highlight=crossvalidator